

CMP project 5

Karmukilan Somasundaram

March 2026

1 Ladder-System

Solving for the Tight binding equation for the given ladder variations we obtain:

$$E(k) = E_0 + 2t \cos(ka) \pm t' \quad (1)$$

$$E_k = E_0 + 2t \cos(ka) \pm (t'^2 + t''^2 + 2t't'' \cos(ka))^{\frac{1}{2}} \quad (2)$$

$$E_k = E_0 + 2t \cos(ka) \pm (t' + 2t'' \cos(ka)) \quad (3)$$

Observe for all three cases we will set $E_0 = 0, t = 1$ for convenience; Without loss of generality keep $a=1$. Note that transitions occurs when $E_{min}^+ = E_{max}^-$. Hence our study of transition points in this context simplifies.

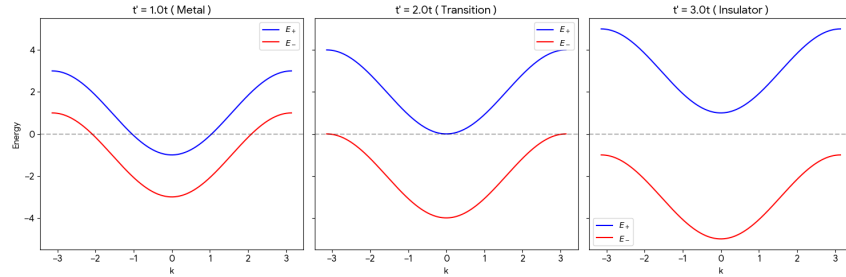


Figure 1: Electron Band structure for question 1

We can see the transition occurs as $t' \rightarrow 2t$. Let's see how the variations affect the transitions for $|t''| < |t'|, |t|$. Observe that t'' makes the band gap smaller in the edges of the Brillouin Zone and bigger in the middle.

Observe the transition is delayed by the introduction of t'' . The material stays in the metal phase for a longer time for both the variations of the parallel ladder. Let's take a look at the plots for different t''

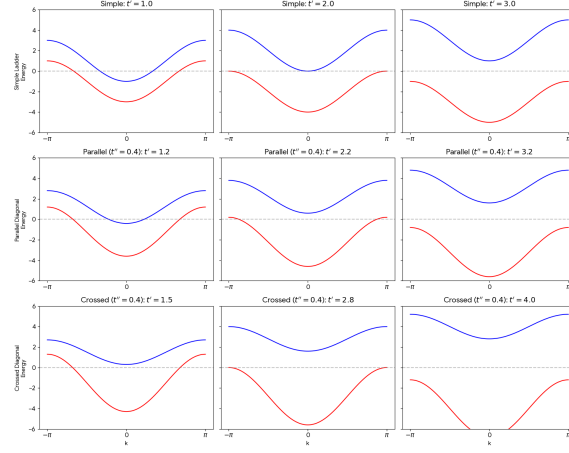


Figure 2: Each row corresponds to a ladder system, the first row is the simple, the second row is the single diagonal hopping and the third row is the double diagonal hopping. $t''=0.4$

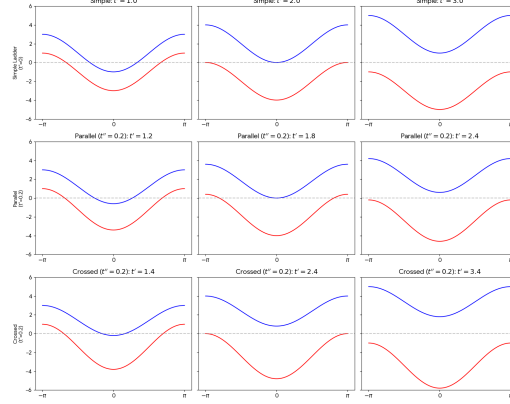


Figure 3: $t''=0.2$

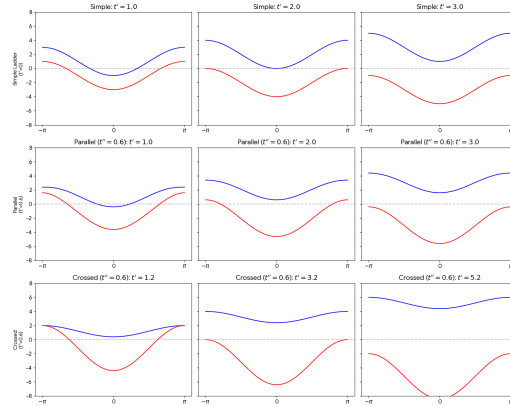


Figure 4: $t''=0.6$

2 Square Lattice

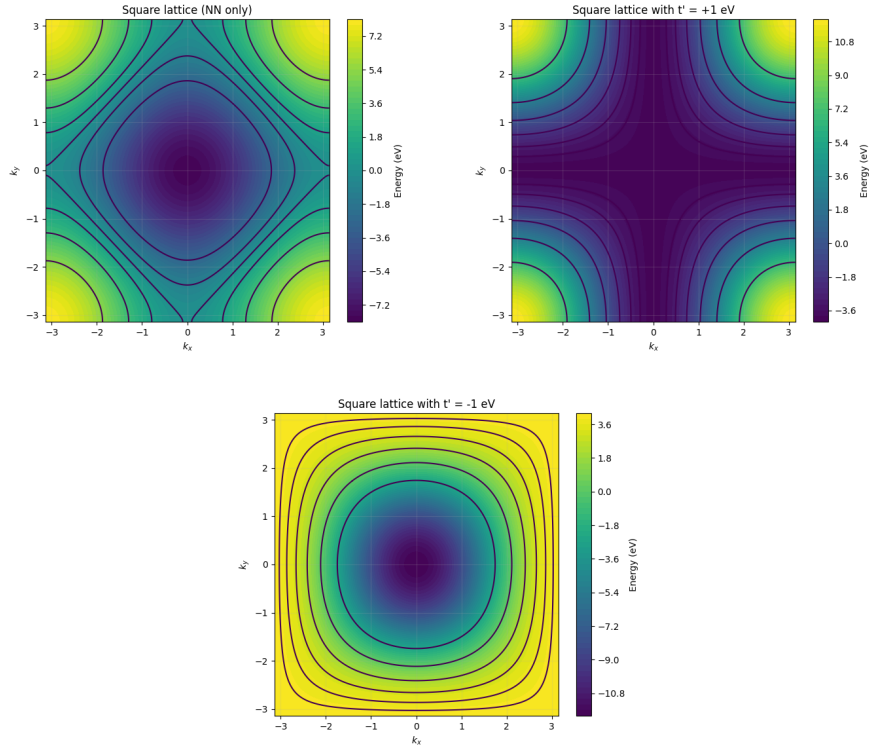


Figure 5: Square lattice a. Nearest Neighbor Hopping b. Diagonal hopping ($t' = 1\text{eV}$) c. Diagonal Hopping ($t' = -1\text{eV}$)

3 Python Code

Listing 1: Ladder system

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # --- CONFIGURATION ---
5 t = 1.0          # Intra-leg hopping
6 t_double = 0.6  # Diagonal hopping (Change this to 0.2, 0.4, 0.8, etc.)
7 k = np.linspace(-np.pi, np.pi, 500)
8
9 def get_bands(tp, tpp, mode):
10     """Calculates  $E_+$  and  $E_-$  for the given ladder mode."""

```

```

11     e_base = -2 * t * np.cos(k)
12     if mode == 'Simple':
13         return e_base + tp, e_base - tp
14     elif mode == 'Parallel':
15         #  $E = -2t \cos(k) \pm \sqrt{tp^2 + tpp^2 + 2*tp*tpp*\cos(k)}$ 
16         coupling = np.sqrt(tp**2 + tpp**2 + 2*tp*tpp*np.cos(k))
17         return e_base + coupling, e_base - coupling
18     else: # Crossed
19         #  $E = -2t \cos(k) \pm (tp + 2*tpp*\cos(k))$ 
20         coupling = tp + 2 * tpp * np.cos(k)
21         return e_base + coupling, e_base - coupling
22
23 # --- PLOTTING ---
24 fig, axes = plt.subplots(3, 3, figsize=(15, 12), sharey=True)
25
26 # Define the t' values for each row to capture the transition
27 # Transition roughly occurs when tp covers the bandwidth
28 tp_values = {
29     'Simple': [1.0, 2.0, 3.0],
30     'Parallel': [1.0, 2.0, 3.0],
31     'Crossed': [1.2, 2*(t + t_double), 5.2] # Middle value is exact
32     transition
33 }
34
35 modes = ['Simple', 'Parallel', 'Crossed']
36
37 for row, mode in enumerate(modes):
38     for col, tp in enumerate(tp_values[mode]):
39         ax = axes[row, col]
40         ep, em = get_bands(tp, t_double if mode != 'Simple' else 0, mode)
41
42         ax.plot(k, ep, color='blue', label='$E_+$ (Anti-bonding)')
43         ax.plot(k, em, color='red', label='$E_-$ (Bonding)')
44
45         # Formatting
46         ax.axhline(0, color='gray', linestyle='--', alpha=0.5)
47         ax.set_ylim(-8, 8)
48         ax.set_title(f"{mode}:  $t' = {tp:.1f}$ ")
49         if col == 0:
50             ax.set_ylabel(f"{mode}\nEnergy  $E/t$ ")
51         if row == 2:
52             ax.set_xlabel("Momentum  $k$ ")
53
54         ax.set_xticks([-np.pi, 0, np.pi])
55         ax.set_xticklabels([' $-\pi$ ', '0', ' $\pi$ '])
56
57 plt.tight_layout()
58 plt.suptitle(f"Ladder Transitions with  $t' = {t_double}$ ", fontsize=16, y
59             =1.02)
60 plt.show()

```

Listing 2: Square Lattice

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # -----
5 # 1. Brillouin zone grid
6 # -----
7 N = 300
8 kx = np.linspace(-np.pi, np.pi, N)
9 ky = np.linspace(-np.pi, np.pi, N)
10 KX, KY = np.meshgrid(kx, ky)
11
12 # -----
13 # 2. Parameters
14 # -----
15 t = -2.0 # eV
16 fillings = np.arange(0.5, 2.01, 0.25)
17
18 # -----
19 # 3. Dispersion function
20 # -----
21 def energy(kx, ky, t, t_prime):
22     return 2*t*(np.cos(kx) + np.cos(ky)) + 4*t_prime*np.cos(kx)*np.cos(ky)
23
24 # -----
25 # 4. Plot function
26 # -----
27 def plot_band_and_fermi(t_prime, title):
28
29     # Compute energy over BZ
30     E = energy(KX, KY, t, t_prime)
31
32     # Flatten and sort once
33     E_sorted = np.sort(E.flatten())
34
35     # Plot band contour
36     plt.figure(figsize=(7, 6))
37     contour = plt.contourf(KX, KY, E, levels=60)
38     plt.colorbar(contour, label="Energy (eV)")
39
40     # Plot Fermi surfaces
41     for n in fillings:
42         frac = n / 2.0 # spin degeneracy
43         E_F = np.quantile(E_sorted, frac)
44
45         plt.contour(
46             KX, KY, E,
47             levels=[E_F],
48             linewidths=1.5,
49             linestyle='solid',

```

```

50         label=f"n={n:.2f}"
51     )
52
53     plt.title(title)
54     plt.xlabel(r"$k_x$")
55     plt.ylabel(r"$k_y$")
56     plt.xlim(-np.pi, np.pi)
57     plt.ylim(-np.pi, np.pi)
58     plt.grid(alpha=0.2)
59
60     plt.show()
61
62     # -----
63     # 5. Run cases
64     # -----
65
66     # (a) Nearest neighbour only
67     plot_band_and_fermi(0.0, "Square_lattice_(NN_only)")
68
69     # (b) With diagonal hopping
70     plot_band_and_fermi(1.0, "Square_lattice_with_t'=+1_eV")
71     plot_band_and_fermi(-1.0, "Square_lattice_with_t'=-1_eV")

```